Generative Adversarial Networks

Learn a generator through competition with a discriminator or critic. Stability depends on balanced optimization, appropriate objectives, and careful handling of gradients and conditioning.

Original study notes based on David Foster, Second edition, 2023. Book pp. 95–128. Source APIs, services, experiments, and model comparisons reflect the book's publication period. These are explanatory summaries, not verbatim reproductions or a claim that every source detail is retained.

Latent noise → Generator → Real/fake comparison → Critic gradients → Update alternating networks

Detailed Notes

GAN and DCGAN architecture

Source: book pp. 95–98 / PDF p. 123

A generator transforms latent noise into candidate images; a discriminator learns to distinguish candidates from real observations. The generator learns through the discriminator's feedback rather than a fixed pixel target paired with each random latent vector.

The DCGAN project uses brick images to expose texture generation and convolutional design. Normalize images to the generator's output range and inspect samples early. After training, generation needs only the generator; discriminator quality is a training instrument, not the final product.

The discriminator and generator

Source: book pp. 99–103 / PDF p. 127

The discriminator downsamples images with convolutional layers and produces a real/fake score. The generator expands a low-dimensional vector into spatial features, then upsamples to the image dimensions. Leaky activations, normalization, and output activation must match the chosen architecture.

Transposed convolutions can create checkerboard artifacts when overlap is uneven; alternative upsampling schemes or compatible kernel/stride choices can help. Ensure the discriminator receives identically preprocessed real and generated data so trivial scaling artifacts do not dominate its decision.

Training the DCGAN

Source: book pp. 104–109 / PDF p. 132

Alternate updates: train the discriminator on real images and detached generator samples, then train the generator through a discriminator whose trainable weights are not updated in that step. Gradients must still flow through the discriminator to the generator.

> Classical objective: $\min_G \max_D E[\log D(x)] + E[\log(1-D(G(z)))]$
> Common generator surrogate: $-E[\log D(G(z))]$

Implementation conventions can flip labels/signs, so derive the loss from the meaning of the output. Verify the parameters updated in each phase, especially with compiled models or custom training loops.

Analysis and training tips

Source: book pp. 110–113 / PDF p. 138

Watch for mode collapse, where many latent points yield similar samples; vanishing gradients from a dominant discriminator; oscillation; and sensitivity to learning rate or update frequency. Fixed latent seeds make progress comparisons meaningful.

Label smoothing/noise, architecture choices, optimizer settings, update ratios, and normalization can stabilize particular runs, but are not universal fixes. A discriminator accuracy near 50% may indicate good competition or simply a broken discriminator. Evaluate generated quality and diversity, not loss curves alone.

Wasserstein loss and Lipschitz constraints

Source: book pp. 114–116 / PDF p. 142

WGAN replaces probabilistic binary discrimination with a critic whose score estimates distributional separation under a Lipschitz restriction. The critic output is not a probability; do not force a sigmoid or interpret it as calibrated realness.

> Critic minimization loss = $E[C(\text{fake})] - E[C(\text{real})]$
> Generator loss = $-E[C(\text{fake})]$

The Wasserstein formulation requires control of the critic's gradients. Early weight clipping crudely enforces a restriction but can damage capacity and optimization. The sign convention above assumes higher critic scores for real data; reversing score meaning reverses corresponding signs.

Gradient penalty and WGAN-GP training

Source: book pp. 117-121 / PDF p. 145

Sample points between real and generated images and penalize critic input-gradient norms that depart from one. This is a practical regularizer for the Lipschitz requirement, not proof of global enforcement everywhere.

$$\hat{x} = a\,x_{real} + (1-a)x_{fake}$$

$$Penalty = \lambda\, E[(\left|\left|\nabla_{\hat{x}} C(\hat{x})\right|\right|_2 - 1)^2]$$

Add the penalty to critic loss and often perform several critic steps per generator step. Differentiate scores with respect to interpolated inputs as well as model weights. Keep per-example gradients meaningful; normalization that couples examples can complicate this calculation. Inspect face samples and diversity alongside critic diagnostics.

Conditional GANs

Source: book pp. 122-128 / PDF p. 150

A conditional GAN supplies a label or other condition to both generator and discriminator/critic. The generator learns p(x|condition); the critic checks whether an image is plausible for that condition, not just plausible overall.

Concatenate or embed labels appropriately for latent vectors and spatial features. During training, pair real images with their true labels and generate matching conditioned examples. Test every class, balanced coverage, and whether changing the condition actually changes output. Conditioning enables control but does not guarantee semantic fidelity or fairness.

Revision Check

1. Which weights update during each GAN phase?
2. Why is a WGAN critic not a probability classifier?
3. Where is gradient penalty calculated?
4. How can loss curves hide mode collapse?
5. Why must the discriminator see the condition too?

Adversarial training learns a moving objective. Track sample quality, diversity, and gradient behavior instead of trusting a single loss or accuracy.

Source links refer to the supplied PDF. Every contents-page topic is mapped in coverage.md. Consult the book for complete implementations, figures, exercises, citations, and experiment evidence.